

Global Engineering Solutions

FEST®

FEST Library Reference Manual

1. SetVersionInfo.....	page 3
2. GetSystemName.....	page 4
3. GetSeqName.....	page 5
4. GetHistoryPath.....	page 6
5. ReadArgument.....	page 7
6. ReadParamaeter.....	page 8
7. ReadEventMsg.....	page 9
8. WaitAbort.....	page 10
9. CheckAbort.....	page 11
10. ResetAbort.....	page 12
11. SystemAbort.....	page 13
12. SystemAbortEx.....	page 14
13. SystemAbortLocal.....	page 15
14. SystemShutDown.....	page 16
15. GetIndexFromObject.....	page 17
16. At.....	page 18
17. DioAt.....	page 19
18. AioAt.....	page 20
19. SioAt.....	page 21
20. SeqAt.....	page 22
21. AlmAt.....	page 23
22. ForEachObj.....	page 24
23. ReplaceObjectName.....	page 25
24. WaitKill.....	page 26
25. CheckKill.....	page 27
26. SeqDebugLog.....	page 28
27. SeqDebugLogV.....	page 29
28. FindCIMObject.....	page 30
29. FindDrv.....	page 31
30. FindEnum.....	page 32
31. FindLookup.....	page 33
32. FindDIO.....	page 34
33. FindAIO.....	page 35
34. FindSIO.....	page 36
35. FindSeq.....	page 37
36. FindAlarm.....	page 38



37. FindAlarmByID.....	page 39
38. RegistAlarmModule.....	page 40
39. RecvAlarmEvent.....	page 41
40. ReadLicenseInfo.....	page 42
41. OnLoadComplete.....	page 43
42 OnInitComplete.....	page 44
43. SeqMain.....	page 45
44. OnUnLoad.....	page 46
45. OnEventRecv.....	page 47
46. SetFunctionHandler.....	page 48

1. SetVersionInfo

● Description

- Sequence의 Version/Build Date/Build Time을 설정하는 함수 이다.
- Sequence Control Manager에 설정한 정보가 display 된다.
- Sequence의 OnLoadComplete()함수에서 설정하면 된다.

● Syntax

- void SetVersionInfo(const TCHAR* szVersion , const TCHAR* szDate , const TCHAR* szTime);

● Parameter

- szVersion ; 버전 정보 입력
- szDate : 컴파일 된 날짜 입력
- szTime : 컴파일 된 시간 입력

● Return

- Nothing

● Sample

```
#define VERSION_INFO      "1.0.35.0"  
#define BUILD_DATE       "12/08/2011"  
#define BUILD_TIME       "14:55:47"
```

```
void OnLoadComplete( const TCHAR* szArgument ) {  
    SetVersionInfo( _TEXT(VERSION_INFO) , _TEXT(BUILD_DATE) , _TEXT(BUILD_TIME) );  
}
```

2. GetSystemName

● Description

- System Name read 함수
- System Name은 Startup.cfg에서 정의 한다
(예, DEMO라고 정의 : SYSTEM DEMO)

● Syntax

- TCHAR* GetSystemName();

● Parameter

- Nothing

● Return

- system name

● Sample

```
printf("GetSystemName : %s \n" , GetSystemName() );
```

3. GetSeqName

- **Description**

- Sequence Name read 함수

- **Syntax**

- TCHAR* GetSeqName();

- **Parameter**

- Nothing

- **Return**

- sequence name

- **Sample**

- ```
printf(" GetSeqName : %s\n" , GetSeqName());
```

---

## 4. GetHistoryPath

---

### ● Description

- History Path Read 함수
- History Path은 Startup.cfg에서 정의 한다  
(예, History라고 정의 : HISTORY History)

### ● Syntax

- TCHAR\* GetHistoryPath();

### ● Parameter

- Nothing

### ● Return

- history path

### ● Sample

```
printf("GetHistoryPath : %s\n" , GetHistoryPath());
```

---

## 5. ReadArgument

---

- **Description**

- Sequence 등록시 정의한 Argument를 Read하는 함수

- **Syntax**

- TCHAR\* ReadArgument();

- **Parameter**

- Nothing

- **Return**

- 정의된 argument

- **Sample**

- ```
printf(" ReadArgument : %s\n" , ReadArgument() );
```

6. ReadParamaeter

● Description

- Sequence Main()에 전달된 Command and Parameter를 Read하는 함수

● Syntax

- TCHAR* ReadParamaeter(void*& pParam , int &nNumOfBytesRecved);

● Parameter

- pParam: 전달 받은 parameter
- nNumOfBytesRecved : parameter byte 수

● Return

- 전달 받은 command

● Sample

```
printf(" ReadCommand : %s\n" , ReadParamaeter(pParam , nNumOfBytesRecved) );
```

7. ReadEventMsg

● Description

- Sequence OnEventRecv()에 전달된 Command and Parameter를 Read하는 함수

● Syntax

- TCHAR* ReadEventMsg(void*& pParam , int &nNumOfBytesRecved);

● Parameter

- pParam: 전달 받은 event parameter
- nNumOfBytesRecved : event parameter byte 수

● Return

- 전달 받은 event command

● Sample

```
printf(" ReadCommand : %s\\n" , ReadEventMsg(pParam , nNumOfBytesRecved) );
```

8. WaitAbort

● Description

- Sequence에 Abort가 전달 되었는지 설정시간 동안 기다리면서 check 하는 함수
- 설정시간 이전에 Abort가 전달되면 바로 TRUE 를 return함

● Syntax

- BOOL WaitAbort(int nWaitTime);

● Parameter

- nWaitTime : waiting time(msec)

● Return

- True : abort
- False : not abort

● Sample

```
While (TRUE) {  
    if(WaitAbort(500)) return SEQ_ABORT;  
}
```

9. CheckAbort

- **Description**

- Sequence에 Abort가 전달 되었는지 check 하는 함수

- **Syntax**

- BOOL CheckAbort();

- **Parameter**

- Nothing

- **Return**

- True : abort
- False : not abort

- **Sample**

```
While (TRUE) {  
    _sleep(500);  
    if(CheckAbort ()) return SEQ_ABORT;  
}
```

10.ResetAbort

- **Description**

- Sequence에 Abort를 전달 받은 것을 reset 하는 함수

- **Syntax**

- void ResetAbort();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

```
While (TRUE) {  
    _sleep(500);  
    if(CheckAbort ()) { ResetAbort(); }  
    if(WaitKill())      return SEQ_ABORT;  
  
}
```

11. SystemAbort

- **Description**

- 모든 Sequence에 Abort를 전달 하는 함수.

- **Syntax**

- void SystemAbort();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

SystemAbort();

12. SystemAbortEx

● Description

- 자신을 제외한 모든 Sequence에 Abort를 전달 하는 함수

● Syntax

- void SystemAbortEx();

● Parameter

- Nothing

● Return

- Nothing

● Sample

SystemAbortEx();

13. SystemAbortLocal

- **Description**

- Only Local Sequence에 Abort를 전달 하는 함수(Remote Sequence에는 전달 하지 않음)

- **Syntax**

- void SystemAbortLocal();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

- SystemAbortLocal();

14. SystemShutDown

- **Description**

- 모든 Sequence를 Kill 하고 System Exit(종료) 하는 함수

- **Syntax**

- void SystemShutDown();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

SystemShutDown();

15. GetIndexFromObject

- **Description**

- Sequence에 등록된 object에 대한 enum index을 찾는 함수 이다
(object=CDIO/CAIO/CSIO/CSequence/CAAlarm)

- **Syntax**

- int GetIndexFromObject(CSeqObject* pObject);

- **Parameter**

- CSeqObject* pObject : 등록된 object 포인터

- **Return**

- object enum 값

- **Sample**

```
int index=GetIndexFromObject( &GATE_OPEN_SNS );
```

16.At

● Description

- Sequence에 등록된 object에 대한 IO를 enum 으로 정의한 값으로 object를 찾는 함수

● Syntax

- CSeqObject* At(int nEnum);

● Parameter

- nEnum : IO 의 Enum 값

● Return

- Sequence object pointer

● Sample

```
CSeqObject *pObj;  
int d=0 , CS;  
pObj = At(eCM1_C01_Wafer + 1);  
d = ( (CDIO*)pObj )->Read(CS);
```

17. DioAt

● Description

- Sequence에 등록된 object 중에 특정 Digital IO로 부터 상대적인 위치의 object를 찾는 함수

● Syntax

- `CDIO* DioAt( CSeqObject* pBegin , int nOffset );`

● Parameter

- pBegin : 정의된 Digital IO object 포인터
- nOffset : pBegin IO 로부터의 상대적 위치 값

● Return

- digital object pointer

● Sample

```
int CS;  
Int iData;  
iData=DioAt( &Lum1PumpingSts , 1 )->Read(CS);
```

Lum1PumpingSts -> 정의된 Digital IO object

18. AioAt

● Description

- Sequence에 등록된 object 중에 특정 Analog IO로 부터 상대적인 위치의 object를 찾는 함수

● Syntax

- CAIO* AioAt(CSeqObject* pBegin , int nOffset);

● Parameter

- pBegin : 정의된 Analog IO object 포인터
- nOffset : pBegin IO 로부터의 상대적 위치 값

● Return

- analog object pointer

● Sample

```
int CS;  
double dData;  
dData=AioAt ( &Lum1PumpPressure , 1 )->Read(CS);
```

Lum1PumpPressure -> 정의된 Analog IO object

19. SioAt

● Description

- Sequence에 등록된 object 중에 특정 String IO로 부터 상대적인 위치의 object를 찾는 함수

● Syntax

- CSIO* SioAt(CSeqObject* pBegin , int nOffset);

● Parameter

- pBegin : 정의된 String IO object 포인터
- nOffset : pBegin IO 로부터의 상대적 위치 값

● Return

- string object pointer

● Sample

```
int CS;  
char szName[256];  
szName =SioAt ( &Lum1PumpName , 1 )->Read(CS);
```

Lum1PumpName -> 정의된 String IO object

20. SeqAt

● Description

- Sequence에 등록된 object 중에 특정 Sequence 로 부터 상대적인 위치의 object를 찾는 함수

● Syntax

- CSequence* SeqAt(CSeqObject* pBegin , int nOffset);

● Parameter

- pBegin : 정의된 sequence object 포인터
- nOffset : pBegin IO 로부터의 상대적 위치 값

● Return

- sequence object pointer

● Sample

```
SEQ_STATUS SeqResult;
```

```
SeqResult =SeqAt( &seqPort1Control , 1 )->Status( );
```

```
seqPort1Control -> Sequence object
```

21. AlmAt

● Description

- Sequence에 등록된 object 중에 특정 Alarm 으로 부터 상대적인 위치의 object를 찾는 함수

● Syntax

- CAlarm* AlmAt(CSeqObject* pBegin , int nOffset);

● Parameter

- pBegin : 정의된 alarm object 포인터
- nOffset : pBegin IO 로부터의 상대적 위치 값

● Return

- alarm object pointer

● Sample

```
ALM_RESULT AlmResult;
```

```
AlmResult =AlmAt( &AlmPrCsTimeoutBM1 , 0)->Post();
```

```
AlmPrCsTimeoutBM1 -> Alarm object
```

22. ForEachObj

- **Description**

- not support(not use)

- **Syntax**

- void ForEachObj(CSeqObject* pBegin , CSeqObject* pEnd , fnForEachHandler ObjHandler , long IParam);

- **Parameter**

- **Return**

- **Sample**

23. ReplaceObjectName

● Description

- Sequence에 정의되어 있는 object 문자를 새로운 문자로 바꿔주는 함수
주로 하나의 exe로 여러 개의 Sequence를 생성할 때 유용하다.
- OnLoadComplete() 부분에서 정의하여 사용한다

● Syntax

- void ReplaceObjectName(const TCHAR* szOld , const TCHAR* szNew);

● Parameter

- szOld ; 기존에 정의 되어 있는 문자열
- szNew : 새로 바꿀 문자열

● Return

- Nothing

● Sample

ReplaceObjectName(_TEXT("%0") , _TEXT("PM1"));

24. WaitKill

● Description

- Sequence kill를 설정시간 동안 기다리면서 check 하는 함수

● Syntax

- `BOOL WaitKill( int nWaitTime );`

● Parameter

- `nWaitTime` : waiting time(msec)

● Return

- True : kill
- False : not kill

● Sample

```
while( true ) {  
    if( WaitKill(100) ) break;  
}
```

25. CheckKill

- **Description**

- Sequence Kill를 check 하는 함수

- **Syntax**

- BOOL CheckKill();

- **Parameter**

- Nothing

- **Return**

- True : kill
 - False : not kill

- **Sample**

```
while( true ) {  
    _sleep(100);  
    if( CheckKill () ) break;  
}
```

26. SeqDebugLog

- **Description**

- 콘솔에 내용을 print 하고, History/SEQ_YYYYMM.log 의 위치에 내용을 저장 한다

- **Syntax**

- void SeqDebugLog(const TCHAR* szFormat , ...);

- **Parameter**

- szFormat : 로그에 저장할 내용

- **Return**

- Nothing

- **Sample**

SeqDebugLog(_TEXT("SeqGateControl Open Start"));

27. SeqDebugLogV

● Description

- 콘솔에 내용을 print 하고, History/SEQ_YYYYMM.log 의 위치에 내용을 저장한다

● Syntax

- void SeqDebugLogV(const TCHAR* szFormat , va_list arg);

● Parameter

- szFormat : 로그에 저장할 내용 및 포맷
- arg : list argument

● Return

- Nothing

● Sample

28. FindCIMObject

● Description

- CIM 에서 특정 Object 찾는 함수 이다

● Syntax

```
void* FindCIMObject(const TCHAR*, int& nTy);
```

● Parameter

- TCHAR* : object name
- nTy : object type

● Return

- object pointer

● Sample

```
char buffer[256];
int nTy;
void *test=NULL;
sprintf( buffer , "TEMP_SET" );
test = FindCIMObject (buffer, nTy );
if(test != NULL) {
    if ( nTy==eCDIO ) printf("TEMP_SET Object is DIO\n");
    else if ( nTy==eCAIO ) printf("TEMP_SET Object is AIO\n");
    else if ( nTy==eCSIO ) printf("TEMP_SET Object is SIO\n");
}
else { printf("TEMP_SET Object is not exist\n"); }
```

29. FindDrv

● Description

- Driver Object를 찾는 함수 이다

● Syntax

- void* FindDrv(const TCHAR*);

● Parameter

- TCHAR* : driver object name

● Return

- driver object pointer

● Sample

```
char buffer[256];
void *test=NULL;
sprintf( buffer , "TEMP_DRIVER" );
test = FindDrv(buffer);
if(test != NULL) { printf("TEMP_DRIVER driver exist\n"); }
else { printf("TEMP_DRIVER driver not exist\n"); }
```

30. FindEnum

● Description

- Enum Object을 찾는 함수 이다

● Syntax

- void* FindEnum(const TCHAR*);

● Parameter

- TCHAR* : enum object name

● Return

- enum object pointer

● Sample

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , "eOffOn" );  
test = FindEnum(buffer);  
if(test != NULL) { printf("eOffOn enum exist\n"); }  
else { printf("eOffOn enum not exist\n"); }
```

31. FindLookup

- **Description**

- Lookup Object을 찾는 함수 이다

- **Syntax**

- void* FindLookup(const TCHAR*);

- **Parameter**

- TCHAR* : Lookup object name

- **Return**

- Lookup object pointer

- **Sample**

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , "LookupTest" );  
test = FindLookup(buffer);  
if(test != NULL) { printf("Lookup exist\n"); }  
else { printf("Lookup not exist\n"); }
```

32. FindDIO

● Description

- Digital IO Object을 찾는 함수 이다

● Syntax

- void* FindDIO(const TCHAR*);

● Parameter

- TCHAR* : digital io object name

● Return

- digital io object pointer

● Sample

```
char buffer[256];
void *test=NULL;
sprintf( buffer , "DigitalIOName" );
test = FindDIO(buffer);
if(test != NULL) { printf("DigitalIOName exist\n"); }
else { printf("DigitalIOName not exist\n"); }
```

33. FindAIO

● Description

- Analog IO Object을 찾는 함수 이다

● Syntax

- void* FindAIO(const TCHAR*);

● Parameter

- TCHAR* : analog io object name

● Return

- analog io object pointer

● Sample

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , "AnalogIOName" );  
test = FindAIO(buffer);  
if(test != NULL) { printf("AnalogIOName exist %n");  
else           { printf("AnalogIOName not exist%Nn");
```

34. FindSIO

● Description

- String IO Object을 찾는 함수 이다

● Syntax

- void* FindSIO(const TCHAR*);

● Parameter

- TCHAR* : string io object name

● Return

- string io object pointer

● Sample

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , " StringIOName" );  
test = FindSIO(buffer);  
if(test != NULL) { printf("StringIOName exist\n"); }  
else { printf("StringIOName not exist\n"); }
```

35. FindSeq

● Description

- Sequence Object을 찾는 함수 이다

● Syntax

- void* FindSeq(const TCHAR*);

● Parameter

- TCHAR* : sequence object name

● Return

- sequence object pointer

● Sample

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , "SequenceName" );  
test = FindSeq(buffer);  
if(test != NULL) { printf("SequenceName exist\n"); }  
else { printf("Sequence not exist\n"); }
```

36. FindAlarm

● Description

- Alarm Name로 Alarm Object을 찾는 함수 이다

● Syntax

- void* FindAlarm(const TCHAR*);

● Parameter

- TCHAR* : alarm object name

● Return

- alarm object pointer

● Sample

```
char buffer[256];  
void *test=NULL;  
sprintf( buffer , " AlarmName" );  
test = FindAlarm(buffer);  
if(test != NULL) { printf("AlarmName exist\n"); }  
else { printf("AlarmName not exist\n"); }
```

37. FindAlarmByID

● Description

- Alarm ID(번호)로 Alarm Object를 찾는 함수 이다

● Syntax

- void* FindAlarmByID(int ALID) ;

● Parameter

- ALID : alarm number

● Return

- alarm object pointer

● Sample

```
int ALID=2001;
void *test=NULL;
test = FindAlarmByID(ALID);
if(test != NULL) { printf("ALID 2001 exist\n"); }
else { printf("ALID 2001 not exist\n"); }
```

38. RegistAlarmModule

● Description

- Alarm Post/Recovery 관련 event를 받을 Sequence를 등록 하는 함수 이다.
- OnInitComplete()에서 등록 하여 사용하면 된다.

● Syntax

- void RegistAlarmModule();

● Parameter

- Nothing

● Return

- Nothing

● Sample

```
BOOL OnInitComplete( const TCHAR* szArgument ) {  
    RegistAlarmModule();  
    return TRUE;  
}
```

39. RecvAlarmEvent

● Description

- Alarm Post/Recovery시 발생하는 event read 함수 이다.
(Alarm event를 받기 위해서는 앞장에 있는 RegistAlarmModule()을 이용하여 먼저 등록 해야 함)

● Syntax

- `BOOL RecvAlarmEvent( CAlarm &Alarm , ALM_RESULT &nResult , SYSTEMTIME &stHappenTime );`

● Parameter

- Alarm : Alarm object pointer
- nResult : Alarm Result(ALM_CLEAR/ ALM_IGNORE/ ALM_RETRY/ALM_ABORT/
ALM_ERROR/ALM_FAIL/ALM_POSTED)
- stHappenTime ; 발생 시간

● Return

- True : alarm event receive
- False : alarm event not receive

● Sample

```
CAlarm Alarm;
ALM_RESULT Result;
SYSTEMTIME HappenTime;
char szTime[256];

if ( RecvAlarmEvent( Alarm , Result , HappenTime ) ) {
    sprintf( szTime , "%02d%02d%02d%02d%02d" , HappenTime.wYear%100 , HappenTime.wMonth ,
    HappenTime.wDay , HappenTime.wHour , HappenTime.wMinute , HappenTime.wSecond );
    printf("[%s] AlarmName[%s]\n",szTime, Alarm.GetName() );
}
```

40. ReadLicenseInfo

- **Description**

- Fest License 정보 read 함수

- **Syntax**

- `BOOL ReadLicenseInfo( unsigned char ucLicenseInfo[] );`

- **Parameter**

- ucLicenseInfo : 라이선스 정보

- **Return**

- True : read ok
 - False: read fail

- **Sample**

41. OnLoadComplete

● Description

- 사용자 정의 Sequence Code Part

Sequence Loading시 제일 먼저 수행되는 부분이다.

OnLoadComplete() 가 수행된 후 IO Table에 사용자가 정해놓은 IO의 내용을 Main Engine으로부터 확인하고 사용가능 여부를 통보 받는다. 이때 잘못된 IO Table이나 없는 내용이 있을 경우 이 Sequence은 Loading에 실패하게 된다.

● Syntax

- void OnLoadComplete(const TCHAR* szArgument);

● Parameter

- szArgument : 각 Sequence 가 Memory에 Loading될 때 함께 전달되는 Data를 ARGUMENT라고 하고, Sequence의 void OnLoadComplete(const TCHAR* szArgument) 부분으로 전달된다.

● Return

- Nothing

● Sample

```
void OnLoadComplete( const TCHAR* szArgument ) {  
    SetVersionInfo( _TEXT(VERSION_INFO) , _TEXT(BUILD_DATE) , _TEXT(BUILD_TIME) );  
    if ( StrCmpNoCase( (char*)szArgument, "PM1") { ReplaceObjectName( "%1" , "PM1" ); }  
    if ( StrCmpNoCase( (char*)szArgument, "PM2") { ReplaceObjectName( "%1" , "PM2" ); }  
  
}
```

42. OnInitComplete

● Description

- 사용자 정의 Sequence Code Part
위의 OnLoadComplete()가 완료되고, IO 정보 확인 후 OnInitComplete()가 수행되게 되며 이 Procedure의 결과가 FALSE일 때는 이 Sequence은 Loading에 실패하게 된다.

● Syntax

- `BOOL OnInitComplete( const TCHAR* szArgument );`

● Parameter

- `szArgument` : 각 Sequence 가 Memory에 Loading될 때 함께 전달되는 Data를 ARGUMENT라고 하고, Sequence의 `BOOL OnInitComplete ( const TCHAR* szArgument )` 부분으로 전달된다.

● Return

- true : Sequence Loading Ok
- false: Sequence Loading Fail

● Sample

```
BOOL OnInitComplete( const TCHAR* szArgument ) {  
    if ( SIMULATION_CONTROL.Equal( eSIMULATION ) ) { SIMULATION_MODE = TRUE; }  
    else { SIMULATION_MODE = FALSE; }  
    return TRUE;  
}
```

43. SeqMain

● Description

- 사용자 정의 Sequence Code Part.
- 다른 Sequence 또는 화면으로 부터 호출한 Run Command가 실행되는 Main 부분 이다

● Syntax

- SEQ_STATUS SeqMain(const TCHAR* szCmd , const void* pParam , int nNumOfBytesRecved);

● Parameter

- szCmd : 도착한 Run Command
- pParam : 도착한 Parameter.
- nNumOfBytesRecved :도착한 Parameter Byte 수

● Return

- SEQ_STATUS(SEQ_UNKNOWN/ SEQ_SUCCESS/SEQ_ABORT/SEQ_ERROR/SEQ_RUNNING/SEQ_FAIL)
- SEQ_SUCCESS: 정상 완료
- SEQ_ABORT: 비정상 완료

● Sample

```
SEQ_STATUS SeqMain( const TCHAR* szCmd , const void* pParam , int nNumOfBytesRecved ) {  
    return SEQ_SUCCESS;  
}
```

44. OnUnLoad

- **Description**

- Sequence가 Kill(종료) 되기 전에 이 함수가 호출 된다.

- **Syntax**

- void OnUnLoad(int nCode);

- **Parameter**

- nCode : unload code

- **Return**

- Nothing

- **Sample**

- ```
void OnUnLoad(int nCode) {
 }
}
```

---

## 45. OnEventRecv

---

### ● Description

- 사용자 정의 Sequence Code Part.
- 다른 Sequence 또는 화면으로부터 호출된 event 가 실행되는 부분이다.

### ● Syntax

- void OnEventRecv( const TCHAR\* szEvt , const void\* pParam , int nNumOfBytesRecv );

### ● Parameter

- szCmd : 도착한 event Command
- pParam : 도착한 event Parameter.
- nNumOfBytesRecv : 도착한 event Parameter Byte 수

### ● Return

- Nothing

### ● Sample

```
void OnEventRecv(const TCHAR* szCommand, const void* pParam , int nNumOfBytesRecv) {
}
```



---

## 46. SetFunctionHandler

---

### ● Description

- CallFunction() 을 사용하기 위하여 OnLoadComplete() 부분에 정의 하여 사용한다.
- 다른 Sequence를 직접 call 하여 변수 등의 값을 읽어 볼 수 있다.

### ● Syntax

- void SetFunctionHandler(fnFunctionMain fnMain);

### ● Parameter

- fnMain:

### ● Return

- Nothing

### ● Sample

```
BOOL FunctionMain(TCHAR *szReturn , int nSize , const TCHAR* szFnName , const TCHAR* szParam ,
void *pParam , int nNumOfBytesRecved);
//
void OnLoadComplete(const TCHAR* szArgument){
 SetFunctionHandler(FunctionMain);
}
```

---

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

